

Projet Tuteuré: Bots culinaire discord

Elazar Cohen

15 mai 2026

Table des matières

1	Introduction	2
2	Présentation du projet et fonctionnalités	2
3	Analyse des besoins	2
4	Choix techniques	2
5	Conception générale du système	3
6	Organisation	3
6.1	A faire	3
6.2	Fait	4
7	Travail fourni et limites du projet	4
7.1	Travail fourni	4
7.2	Limites du projet	6
8	Conclusion	7

1 Introduction

Ce fichier présente mon projet tuteuré pour le deuxième semestre de L3. Le projet est un bot discord dans un premier temps qui propose à un groupe de personnes des recettes personnalisées en fonction du prompt donné par l'utilisateur.

2 Présentation du projet et fonctionnalités

Introduction Le projet consiste à concevoir un bot Discord capable de proposer des recettes personnalisées aux utilisateurs d'un même groupe. Les utilisateurs pourront interagir avec le bot via des commandes textuelles. Contrairement à un simple système de recommandation de recettes, le bot créera des recettes personnalisées à partir des informations sur les utilisateurs : leurs goûts, les instruments de cuisine qu'ils ont à disposition ou encore leur budget. Le bot est conçu pour fonctionner dans un cadre collaboratif, ou pour être adapté à une seule personne.

Objectifs du projet L'objectif principal de ce projet est de développer un système capable de générer des recettes culinaires adaptées aux préférences et contraintes des utilisateurs. Le projet vise à montrer qu'il est possible d'utiliser un modèle de langage pour produire des contenus personnalisés de manière contrôlée, en tenant compte de données utilisateurs structurées. Un autre objectif est de proposer un outil simple d'utilisation, intégré à une plateforme de communication existante, afin de faciliter l'adoption par les utilisateurs.

3 Analyse des besoins

L'utilisation du bot répond à un besoin courant : proposer des idées de repas adaptées à des personnes ayant des goûts et des contraintes différentes. Dans un contexte de groupe, il est souvent difficile de trouver une recette qui convienne à tout le monde. Le bot vise à simplifier cette prise de décision en générant automatiquement des propositions cohérentes avec les préférences exprimées.

Les besoins principaux identifiés sont :

- une personnalisation des recettes
- une prise en compte de plusieurs utilisateurs
- une interaction simple et rapide
- une génération de recettes originales et adaptées.

4 Choix techniques

Langage et environnement Le projet sera développé en Python. Ce langage a été choisi pour sa simplicité, sa lisibilité et son écosystème riche, notamment dans le domaine de l'intelligence artificielle. Python permet également une intégration simple avec des bibliothèques dédiées à la création de bots Discord.

Plateforme Discord Discord a été choisi comme plateforme de déploiement car il s'agit d'un outil largement utilisé pour la communication de groupe. L'intégration du bot dans un serveur Discord permet une interaction naturelle avec plusieurs utilisateurs dans un même espace.

Modèle de langage Le bot s'appuiera sur un modèle de langage exécuté localement. Ce choix permet de garder un contrôle sur le fonctionnement du système et d'éviter la dépendance à des services externes. Le modèle sera utilisé pour générer des recettes à partir d'un contexte fourni par le système.

5 Conception générale du système

Le fonctionnement du bot repose sur plusieurs composants :

- une interface utilisateur via Discord
- un module de gestion des profils utilisateurs
- un module de génération de recettes
- un modèle de langage chargé de produire les recettes.

Lorsqu'un utilisateur effectue une demande, le bot récupère les informations nécessaires, construit un contexte adapté et transmet ces informations au modèle de langage afin de générer une recette personnalisée.

6 Organisation

6.1 A faire

Semaine 1 : 12 février - 19 février

- Organisation et clarification du travail demandé
- Recherche des données
- Création du schéma relationnel et création du dump de la base de donnée relationnel

Semaine 2 : 19 février - 26 février

- Création du bot discord
- Automatisation pour rentrer les données dans la base et vérifications des données
- Automatisation de la recherche des données par index dans la base de donnée
- Téléchargement du LLM locale

Semaine 2 : 19 février - 26 février

- Joindre les 3 parties (données, LLM et Bot)
- Lancement du bot pour avoir des feedbacks
- possiblement finir si je n'ai pas fini une partie

6.2 Fait

Semaine 1 : 12 février - 19 février

- Organisation et clarification du travail demandé
- Recherche et récupération des données
- Création du schéma relationnel
- Création du dump de la base de données relationnelle.

Semaines 2, 3 et 4 : 19 février - 5 mars

- Création du bot Discord
- Début du travail de normalisation des données
- Nettoyage et structuration progressive des données.

Semaines 5, 6 et 7 : 5 mars - 26 mars

- Insertion des données normalisées dans la base de données
- Transformation des recettes en embeddings vectoriels
- Création de la table `recipe_embeddings`
- Étude et utilisation de `pgvector`
- Mise en place de la recherche par similarité cosinus.

Semaines 6 et 7 : 26 mars - 9 avril

- Installation et configuration locale du modèle de langage via Ollama avec Mistral
- Intégration des différentes parties du projet (base de données, LLM et bot Discord)
- Création de la commande `/recette` sur Discord
- Connexion de la commande au programme principal de génération de recettes.

Semaines 8 et 9 : 9 avril - 23 avril

- Déploiement du bot sur un serveur Discord privé afin de récolter des retours utilisateurs
- Amélioration importante des performances d'insertion dans la base de données
- Réduction du temps d'insertion de 10 000 recettes de 2h30 à environ 2 minutes
- Amélioration progressive de la normalisation et de la qualité des données.

Semaine 12 : 11 mai - 15 mai

- changement de façon d'insérer dans la base de donnée en passant par un csv
- rédaction du rapport et du readme
- préparation à la soutenance

7 Travail fourni et limites du projet

7.1 Travail fourni

Fonctionnement général Le bot permet à un utilisateur de demander une recette personnalisée directement depuis Discord à l'aide d'un prompt textuel. Une fois la demande envoyée, le système traite les informations et génère une recette adaptée au contexte fourni

par l'utilisateur. Lorsque le modèle de langage est exécuté localement, le temps de réponse moyen est d'environ deux minutes.

Le projet intègre plusieurs composants fonctionnant ensemble :

- un bot Discord permettant l'interaction avec les utilisateurs ;
- une base de données relationnelle contenant les recettes ;
- un système de recherche vectorielle basé sur les embeddings ;
- un modèle de langage local chargé de générer les recettes.

Fonctionnement interne Lorsqu'un utilisateur effectue une demande, le texte du prompt est transformé en embedding vectoriel. Le système effectue ensuite une recherche par similarité cosinus dans la table contenant les embeddings des recettes afin de récupérer les recettes les plus pertinentes par rapport à la demande.

Les recettes les plus proches sont ensuite transmises au modèle de langage Mistral exécuté localement via Ollama. Le modèle utilise ces recettes comme contexte afin de générer une nouvelle recette cohérente et adaptée à la demande de l'utilisateur.

Une fois la génération terminée, le bot envoie automatiquement la réponse dans Discord et notifie l'utilisateur que sa recette est prête.

Travail réalisé sur les données Une partie importante du projet a concerné le traitement et la normalisation des données. Les recettes récupérées depuis différentes sources possédaient des formats variés et souvent incohérents. Il a donc été nécessaire de nettoyer les données, standardiser les ingrédients et restructurer les informations afin de pouvoir les exploiter correctement dans la base de données.

L'utilisation du parser `ingredient_parser` a permis d'améliorer significativement la qualité de la normalisation des ingrédients et de rendre les données plus homogènes.

Optimisation des performances Une partie importante du travail a porté sur l'optimisation des performances d'insertion des données dans la base de données. Au début du projet, les recettes étaient insérées ligne par ligne directement depuis le programme Python. Cette méthode fonctionnait correctement pour de petites quantités de données, mais elle est rapidement devenue très lente lorsque le nombre de recettes a augmenté.

Chaque insertion provoquait une requête SQL ainsi qu'une transaction indépendante, ce qui entraînait un coût important en temps d'exécution. Avec cette méthode, l'insertion complète d'environ 10 000 recettes pouvait prendre plus de deux heures et demie.

Afin d'améliorer les performances, une première optimisation a consisté à regrouper les insertions par lots. Les recettes étaient alors insérées par batchs d'environ 5 000 lignes afin de réduire le nombre de requêtes et de transactions effectuées. Cette approche a permis une amélioration des performances, mais le temps d'insertion restait encore trop élevé pour une utilisation efficace.

Après plusieurs recherches et discussions avec Monsieur Forax, il est apparu qu'une meilleure solution consistait à utiliser des fichiers CSV afin d'effectuer des insertions massives directement dans PostgreSQL. Le système a donc été entièrement modifié.

Désormais, le programme génère un fichier CSV complet pour chaque table de la base de données. Toutes les lignes sont préparées à l'avance, puis insérées en une seule opération

grâce aux outils d'importation de PostgreSQL. Cette méthode réduit fortement le nombre d'échanges entre Python et la base de données.

Afin de conserver correctement les relations entre les tables, les identifiants sont également générés manuellement lors de la création des fichiers CSV. Cela permet de relier correctement les recettes, les ingrédients et les embeddings sans avoir à effectuer des requêtes supplémentaires pour récupérer les clés générées automatiquement.

Grâce à cette nouvelle méthode, le temps d'insertion d'environ 10 000 recettes est passé de plus de deux heures et demie à environ deux minutes.

Cependant, il est difficile de réduire davantage ce temps de traitement. En effet, la majorité du temps restant est désormais consacrée à la génération des embeddings vectoriels. Cette étape représente environ 90 % du temps total d'insertion des données.

Les embeddings étant générés localement sur le processeur (CPU) et non sur une carte graphique (GPU), les calculs restent relativement coûteux en temps. Les performances pourraient être améliorées avec un GPU dédié ou une infrastructure plus puissante, mais cela dépassait la capacité de mon ordinateur.

7.2 Limites du projet

Fonctionnalités non terminées L'objectif initial du projet était de proposer une personnalisation avancée des recettes en fonction des profils utilisateurs. Le bot devait notamment enregistrer les préférences, allergies, budgets ou équipements de cuisine des utilisateurs afin de générer des recettes adaptées à chaque personne ou à un groupe complet.

Cependant, le temps nécessaire à la mise en place du système de génération et du traitement des données a été plus important que prévu. Certaines fonctionnalités avancées n'ont donc pas pu être entièrement développées avant la fin du projet.

Limites techniques Plusieurs difficultés techniques ont été rencontrées durant le développement :

- la compréhension et la mise en place du fonctionnement du RAG ;
- la normalisation automatique des données issues de différentes sources ;
- la cohérence des recettes générées par le modèle de langage ;
- les performances de traitement et d'insertion dans la base de données ;
- les limitations matérielles liées à l'exécution locale d'un modèle de langage.

Le fonctionnement local du modèle permet de conserver une certaine indépendance vis-à-vis de services externes, mais cela implique également des temps de génération plus élevés et des performances limitées par le matériel disponible.

Axes d'amélioration Plusieurs améliorations pourraient être apportées au projet dans le futur :

- ajout d'un véritable système de profils utilisateurs ;
- prise en compte des allergies et restrictions alimentaires ;
- apprentissage à partir des retours utilisateurs ;
- amélioration de la qualité des recettes générées ;
- optimisation supplémentaire des performances ;
- déploiement sur un serveur dédié afin de réduire les temps de réponse.

8 Conclusion

Ce projet tuteuré a pour objectif de mettre en œuvre un bot Discord capable de générer des recettes personnalisées à l'aide d'un modèle de langage. Il permet d'aborder des notions telles que la personnalisation, la gestion de données utilisateurs et l'utilisation raisonnée de l'intelligence artificielle. Le projet constitue une application concrète des compétences acquises durant la licence informatique.